



Lecture 22: Passing Data with Props

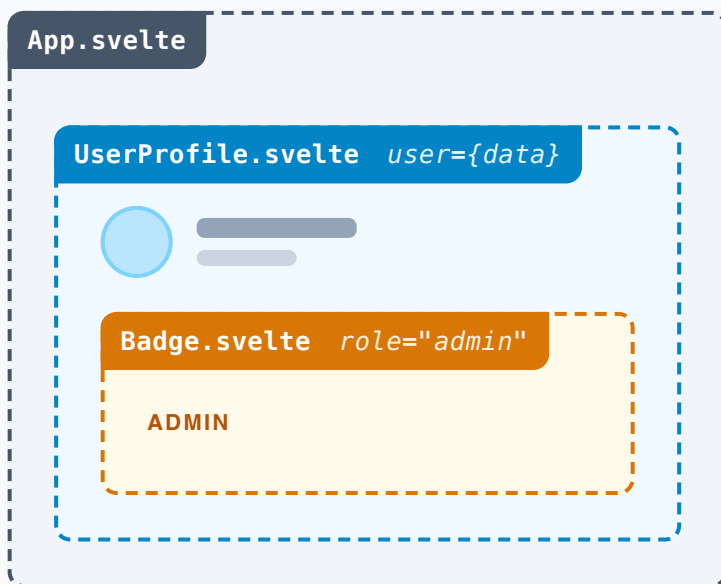
The Component Boundary

You've broken your interface down into pieces. A large `App.svelte` has delegated the job of rendering an individual user to a smaller file named `UserProfile.svelte`. This is the cornerstone of component-driven architecture: isolating responsibility. But isolation comes with a boundary. By default, `UserProfile.svelte` knows absolutely nothing about the data living inside `App.svelte`.

To cross this boundary, we need a controlled bridge. We need a way for the parent (the caller) to hand information down to the child (the callee). In UI frameworks, these inputs are called **props** (short for properties).

The File Structure

Before we write the code, let's look at the physical architecture. Components are just files on your computer. A parent component imports a child component from a folder, creating a hierarchical relationship.



Code Quality Standards

Writing functional code is only the first step. Writing code that is maintainable, predictable, and clean is what separates a novice from a professional. Review the standards below.

1. Prop Mutability

Avoid redefining or forcefully changing props inside the child. It breaks the one-way data flow model, causing unexpected side effects in the parent. If you need formatted data, use a `$derived` state instead.

✗ ANTI-PATTERN

```
let { userCount } = $props();

// ✗ Direct mutation of a received prop
userCount = userCount + 1;

function reset() {
  userCount = 0;
}
```

✓ BEST PRACTICE

```
let { userCount, onReset } = $props();

// ✓ Creating a new variable based on the prop
let displayCount = $derived(userCount * 2);

function handleReset() {
  // Notifying the parent to change its own state
  onReset();
}
```

2. Passing Configuration

Passing highly specific styling booleans couples the parent tightly to the child's internal CSS architecture. Instead, pass semantic intentions (like "danger" or "primary").

✗ ANTI-PATTERN

```
<!-- ✗ Parent is micro-managing child styles -->
<Button
  isRed={true}
  hasShadow={false}
  isLargeText={true}
/>
```

✓ BEST PRACTICE

```
<!-- ✓ Parent passes semantic intent -->
<Button
  variant="danger"
  size="large"
/>
```